

Базовые сущности Kubernetes: Persistent Volume, Persistent Volume Claim

Microservice Architecture

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?





Майоров Антон

Старший разработчик ПО

- 10+ лет опыта разработки ПО на платформе .NET
- Специализация: приложения со сложной бизнес логикой
- Последние 4 года - старший системный разработчик в Ренессанс Страхование

@amaiorov

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе

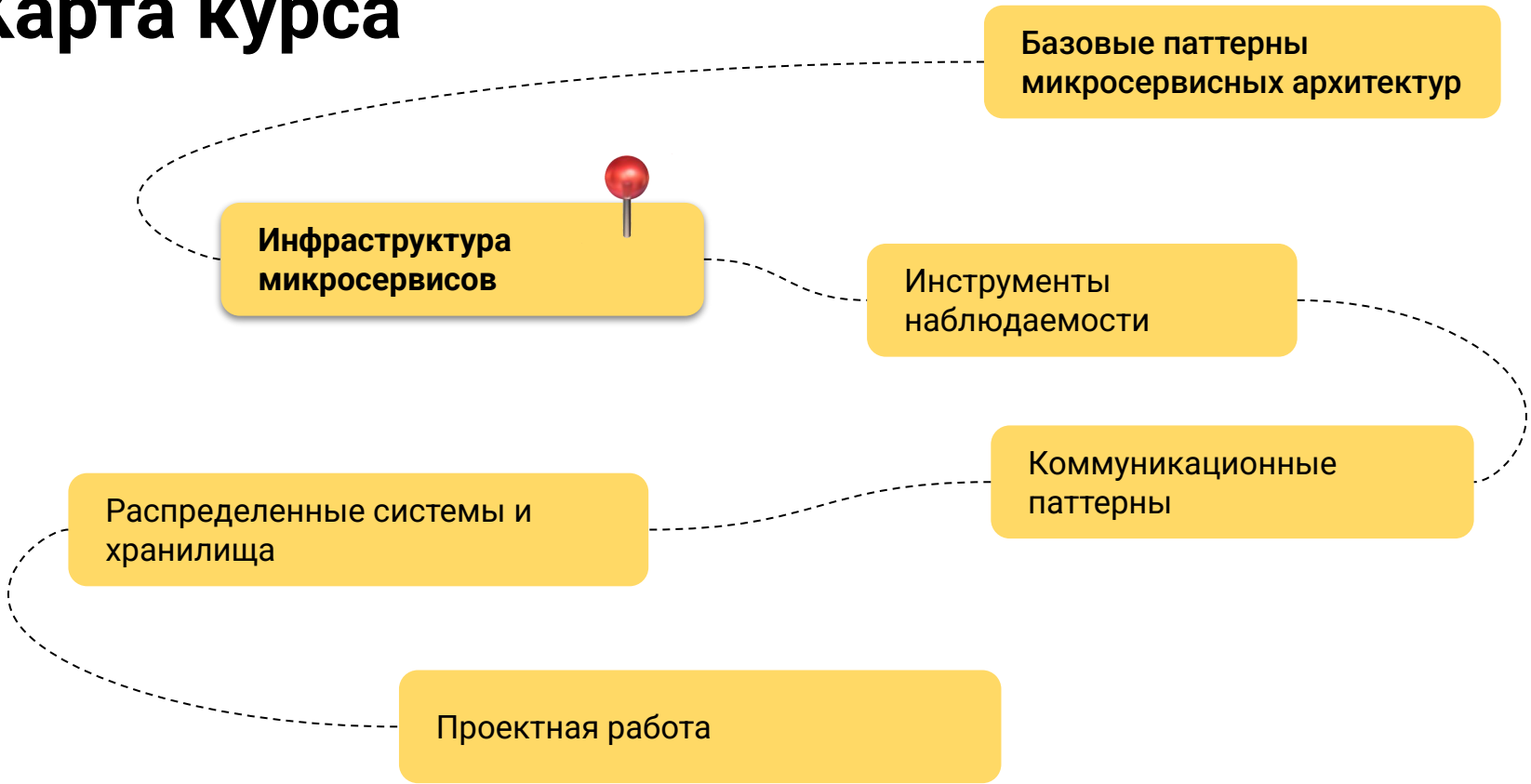


Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса


Базовые паттерны микросервисных архитектур
Инфраструктура микросервисов
Инструменты наблюдаемости
Коммуникационные паттерны
Распределенные системы и хранилища
Проектная работа

Маршрут вебинара

- Persistent Volume
- Persistent Volume Claim
- Подведем итоги

Цели вебинара

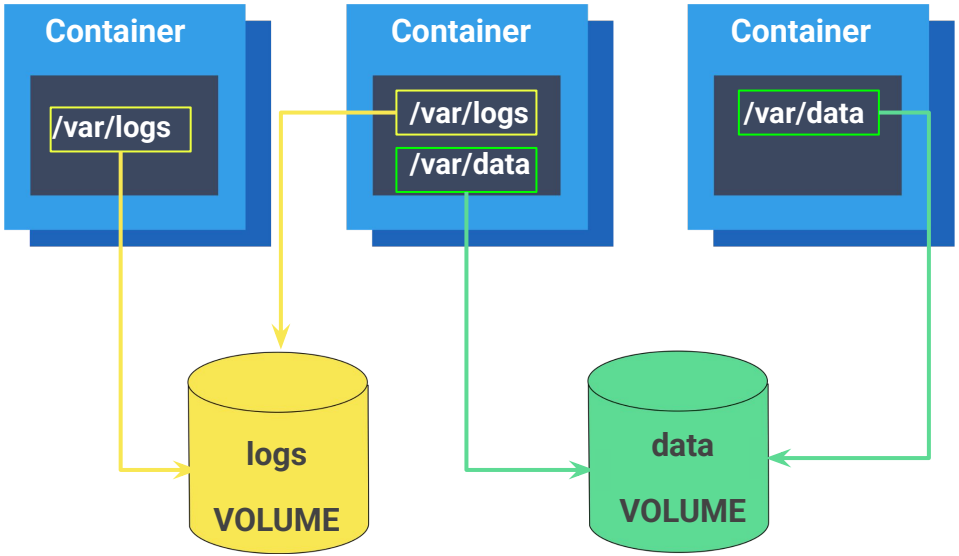
К концу занятия вы сможете

1. Понять особенности монтирования Volumes в поды
2. Создавать и конфигурировать объект k8s PV и PVC

Volumes

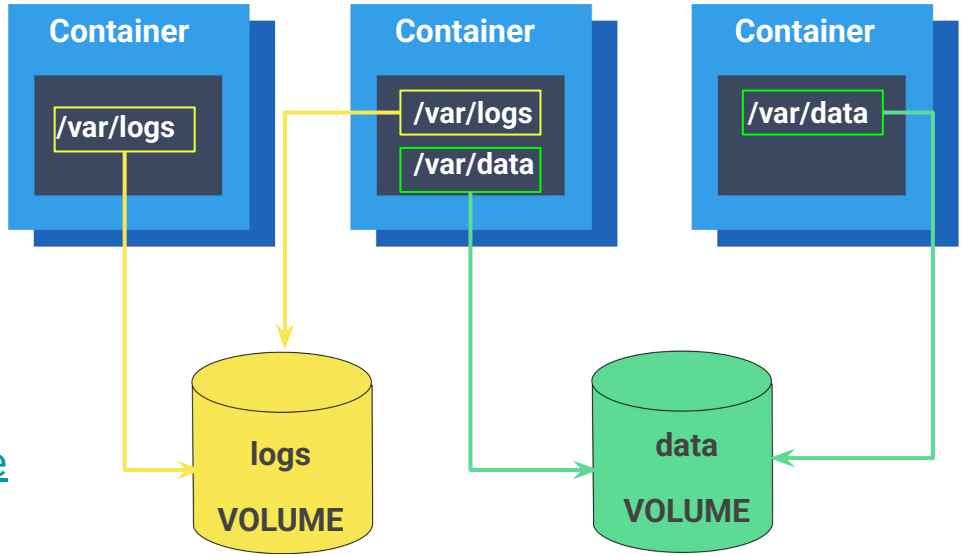
Volumes. Зачем

- Сохранение данных в процессе работы пода
- Sharing данных между подами одного deployment
- Доступ к уже существующим файловым ресурсам, необходимым для работы пода



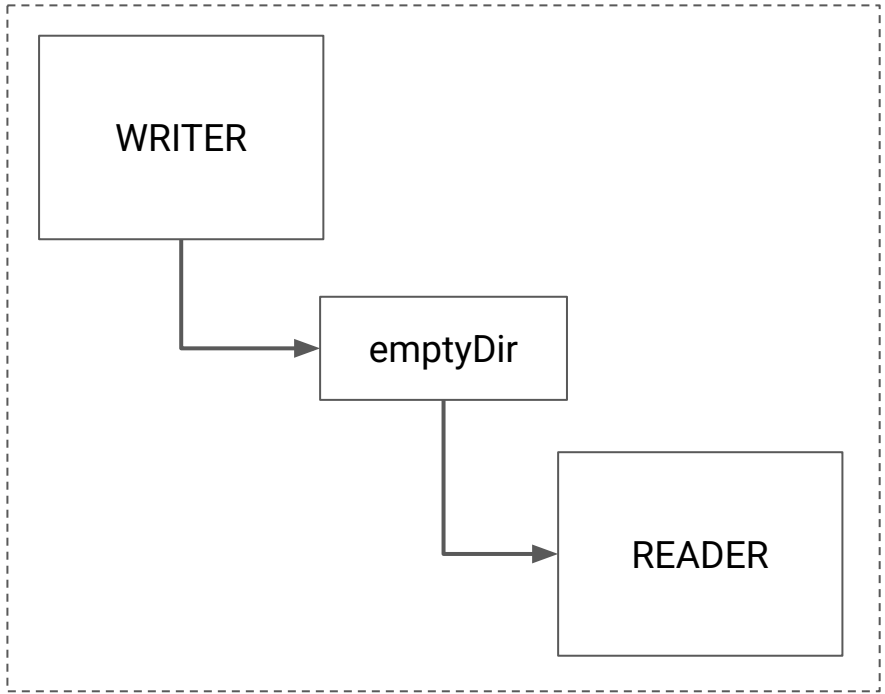
Volumes. Виды

- Ephemeral
  - EmptyDir
  - ConfigMap
- Persistent
  - hostPath
  - Nfs
  - …
  - https://kubernetes.io/docs/concepts/storage/volumes/



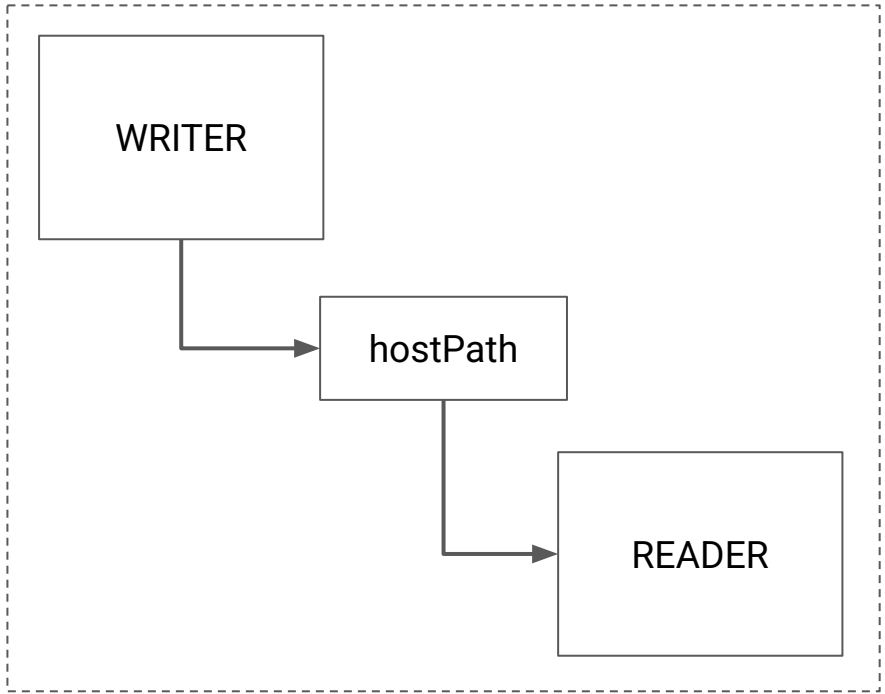
Volumes. emptyDir

```yaml
containers:
  - name: writer
    image: busybox:1.37.0
    command: ["sleep", "86400"]
    volumeMounts:
      - name: data
        mountPath: /cache1
  - name: reader
    image: busybox:1.37.0
    command: ["sleep", "86400"]
    volumeMounts:
      - name: data
        mountPath: /cache2
volumes:
  - name: data
    emptyDir: {}
```



Volumes. hostPath

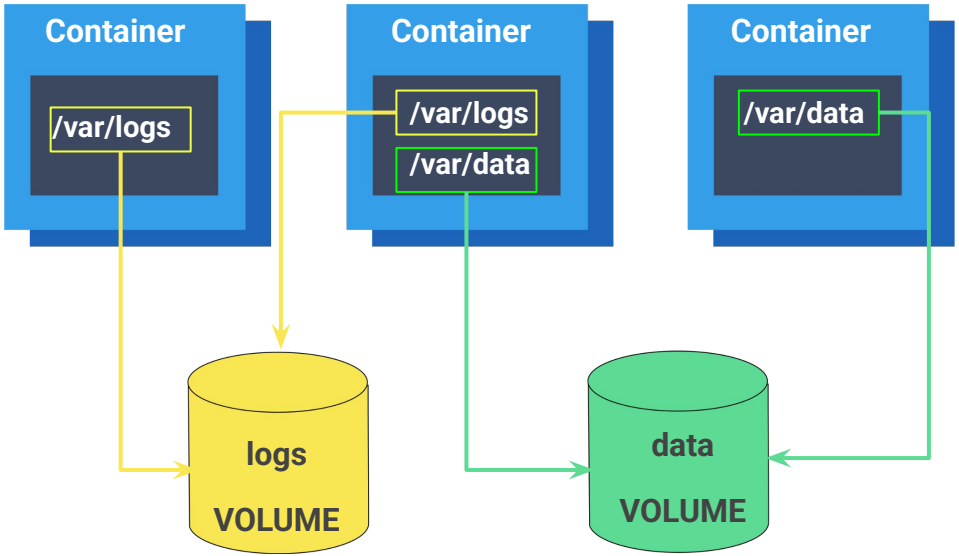
```yaml
containers:
  - name: writer
    image: busybox:1.37.0
    command: ["sleep", "86400"]
    volumeMounts:
      - name: data
        mountPath: /cache1
  - name: reader
    image: busybox:1.37.0
    command: ["sleep", "86400"]
    volumeMounts:
      - name: data
        mountPath: /cache2
volumes:
  - name: data
    hostPath:
      path: /var/lib/pod-cache
      type: DirectoryOrCreate
```



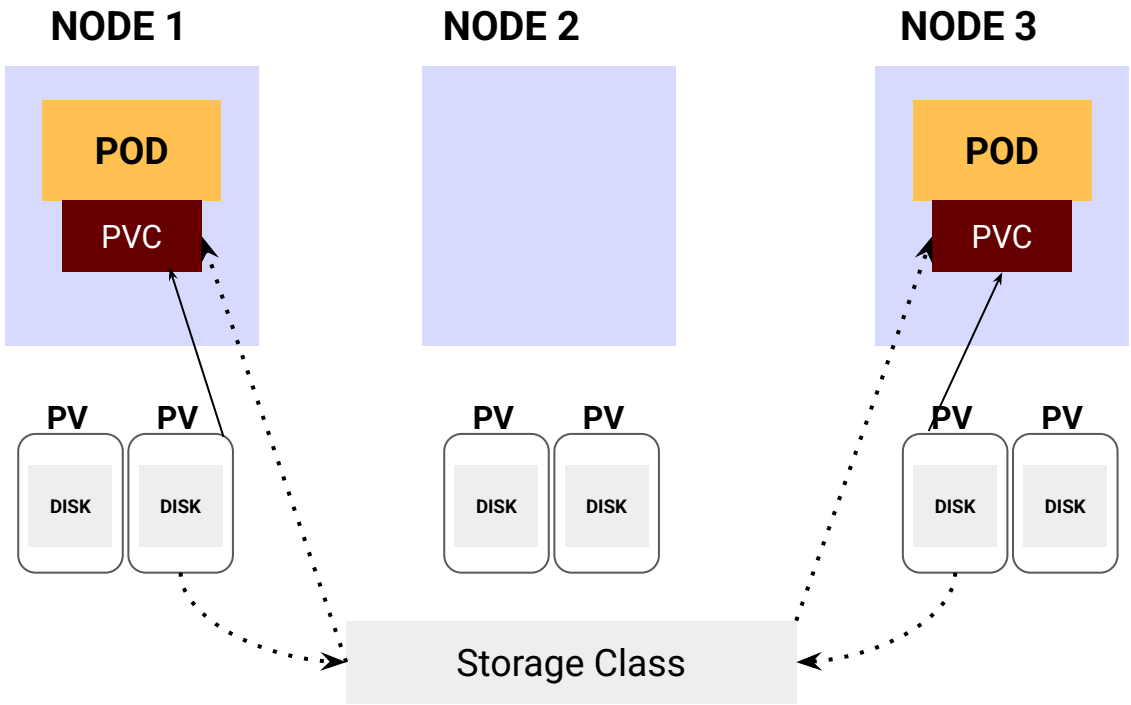
Объекты PV, PVC

Объекты PV, PVC. Зачем

- Предоставление доступа подов к требуемым им хранилищам
- Абстракция предоставления доступа к хранилищу от его реализации



PV, PVC & StorageClass



PersistentVolume. Пример

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0003
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
```

Объект Persistent Volume Claim. Пример

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
  selector:
    matchLabels:
      release: "stable"
    matchExpressions:
      - {key: environment, operator: In, values: [dev]}
```

PersistentVolume. ЖЦ

- Provisioning (Manual / Auto)
- Binding
- Using
- Reclaiming (Retain / Delete)

Вопросы?

+ если есть вопросы

− если вопросов нет

Рефлексия

Вопросы для самопроверки

1. Почему emptyDir переживает перезапуск контейнера, но не замену Pod?
2. Что меняется, когда прямой hostPath заменяют ссылкой на PVC?
3. Можно ли двум PVC выдать по 1 GiB из одного PV на 3 GiB?
4. Почему удаление storageClassName и значение "" дают разный результат?
5. Что автоматизирует StorageClass вместе с provisioner?
6. Удалит ли политика Delete у PV данные сразу после удаления Pod?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️